



Sede Regional Brunca, Campus Pérez Zeledón

Título: Informe de Proyecto 1

Curso: Estructuras de Datos

Profesor: M.C. Pablo Venegas Elizondo

II Ciclo 2026 -NRC 53677

Estudiante:

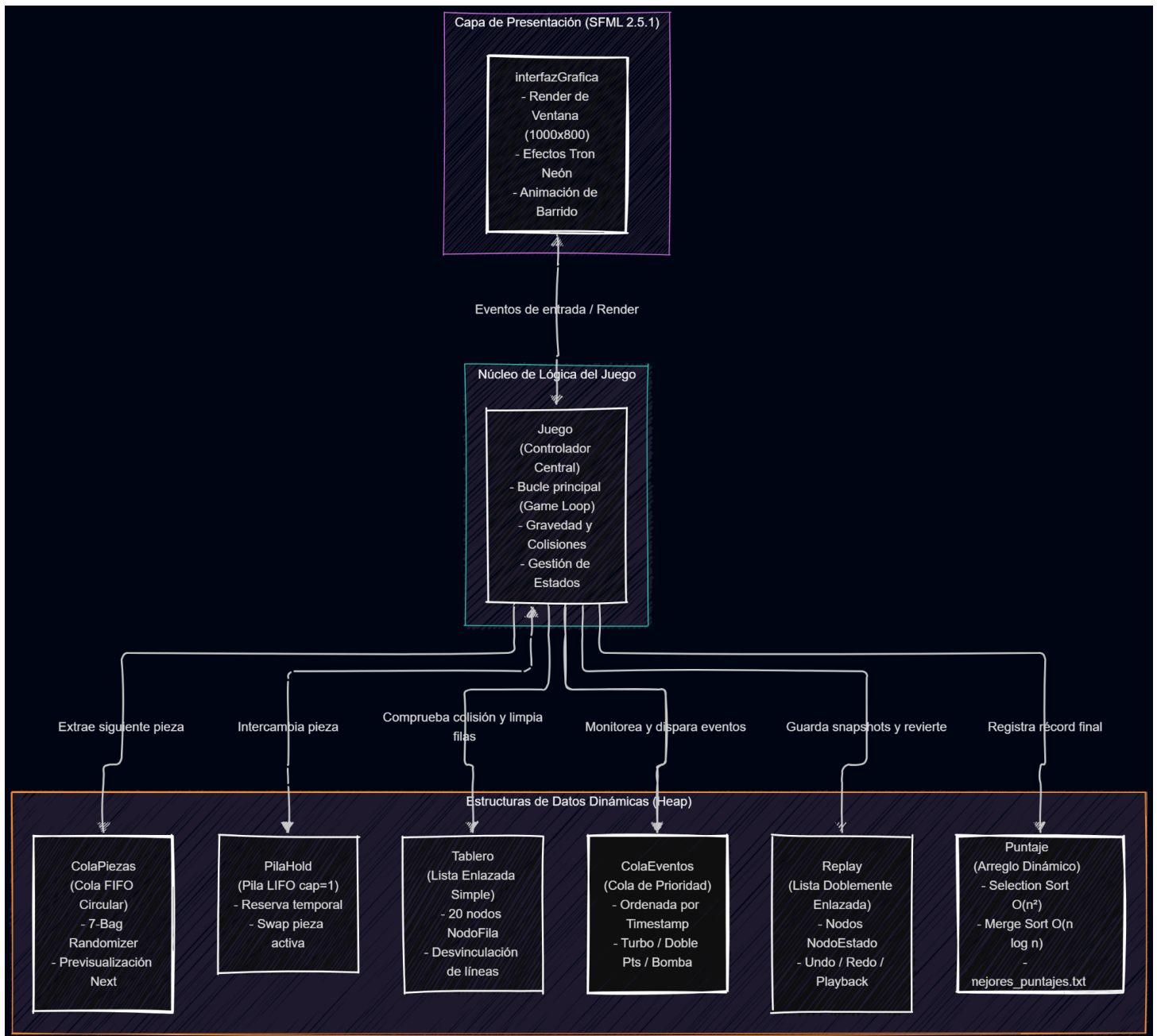
Oscar Salas Fallas

Cedula:

119440628

27 de septiembre del 2026

1. Diagrama y Relación entre Estructuras:



Como se ve en el diagrama anterior, existen distintas capas de clases encargadas de separar todos los procesos del juego, empezando por la parte gráfica encargada de mostrarle al jugador el estado actual del juego además de interpretar la acción siguiente que quiere hacer el usuario, en base a esta información la clase de juego interpreta esas acciones y manipula los distintos contenedores de datos, explicados a continuación:

- **Generar pieza:** El juego solicita a Cola Piezas genera una lista pseudo aleatoria de todas las posibles piezas, en juego se extrae la pieza del frente de la cola y se inserta dentro del tablero en posición inicial, la próxima del frente se coloca en una variable de siguiente para poder visualizarla. Una vez vacía la bolsa esta aplica el mismo proceso de llenado automático.

- **Reserva:** El juego evalúa si ya se realizó una operación de hold este turno, si no se ha hecho revisa si el hold posee alguna pieza dentro, en caso de que si la almacena, inserta la nueva al hold, elimina la pieza actual y genera una nueva pieza actual en base al valor enviado del hold, si no hubiera pieza en hold brinca el paso de almacenar lo que devuelva el hold, inserta la pieza actual igual, elimina la actual y genera una nueva actual con la pieza siguiente.
- **Fijar piezas y Detectar líneas llenas:** Cada pieza se retira del tablero antes de aplicarle cualquier movimiento, una vez retirada se le aplica el nuevo estado y se le envía al tablero para que evalúe si en su posición nueva está libre, en caso de que si, inserta la nueva posición al tablero, en caso de que no esté libre para movimientos laterales simplemente devuelve la pieza a su posición original y la inserta al teclado, si es un movimiento de caída hace el mismo proceso con la adición de liberar esa pieza de la pieza actual, moviendo la siguiente a actual y actualizando la variable de siguiente, luego de eso evalúa si alguna fila se completo para posteriormente eliminarla y agregar una nueva fila al principio.
- **Eventos:** La cola de eventos asegura que los eventos están ordenados del más próximo al más lejano, durante cada iteración del ciclo principal se le envía el tiempo que lleva ejecutándose la partida a la cola de eventos para evaluar si ya corresponde disparar el evento, una vez disparados todos los eventos la misma cola vuelve a llenarse asignando eventos nuevos con tiempos de disparo nuevos y relativos al tiempo actual.
- **Historial:** El historial se genera con cada interacción que haga el usuario, en cada interacción se genera un objeto estado que almacena todo, desde el tablero hasta la cantidad de líneas eliminadas, y luego se almacena una copia de ese estado en el historial para habilitar la reversión de acciones y reproducir toda la partida al final, si el usuario revierte una acción y genera una totalmente nueva el mismo historial se encarga de eliminar cualquier estado posterior al actual para poder ingresar el nuevo estado.
- **Persistencia:** Una vez terminada una partida el sistema pregunta al usuario un nombre para poder almacenar y agregar al registros histórico de puntajes su partida, esta almacena todos los puntajes en la historia del juego, a la hora de leerlos los va ingresando a memoria de manera desordenada para posteriormente ordenarlos segun la eleccion del usuario, ya se un selection sort (simple) o un merge sort (eficiente).

2. Tabla de Complejidad:

Estructura de Datos	Operación	Mejor Caso	Caso Promedio	Peor Caso	Complejidad Espacial
Tablero (Lista enlazada)	Inserción fila vacía	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Tablero (Lista enlazada)	Eliminar Fila Completa	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Tablero (Lista enlazada)	Detectar líneas completas en todo el tablero	$O(\text{Filas} \times \text{Columnas})$	$O(\text{Filas} \times \text{Columnas})$	$O(\text{Filas} \times \text{Columnas})$	$O(\text{Filas} \times \text{Columnas})$
Cola Piezas(Cola FIFO)	Encolar nueva/llevar bolsa	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Cola Piezas(Cola FIFO)	Desencolar	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Pila Hold(Pila LIFO Única)	Guardar/intercambiar	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Cola Eventos (Prioridad)	inserción ordenada	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Cola Eventos (Prioridad)	Desencolar/Disparar	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Replay (Doble)	Insertar Final	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Replay (Doble)	Retroceder/Avanzar	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Puntaje (Ordenamiento)	SelectionSort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$
Puntaje (Ordenamiento)	Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

3. Preguntas de análisis:

- Compare el tiempo de ordenar la tabla de puntajes con el algoritmo de complejidad $O(n^2)$ contra el de $O(n \log n)$, para tamaños crecientes de datos de prueba. ¿A partir de qué tamaño se nota la diferencia? Coincide con lo que predice la notación asintótica?
R/ Empecé a notar la diferencia cuando se llegó a registros altos como los 1000, en donde en mi equipo se representó una diferencia de 0.991ms, es poco pero ahí empezó el cambio, para la simulación de los 10.000 registros llegó a haber una diferencia de hasta 53 ms, estas simulaciones demuestran muy bien la diferencia asintótica que existe entre ambos algoritmos, mostrando una eficiencia de hasta 86% en el merge a comparación del selection.

- Explique por qué representar el tablero como una lista enlazada de filas es una forma razonable de modelar la limpieza de líneas, y que costo (en operaciones) tiene en su implementación insertar una fila vacía y eliminar una fila completa.

R/ El utilizar una lista permite hacer operaciones mucho más sencillas, como cada fila posee internamente un arreglo que representa sus celdas estamos generando una estructura de datos bidimensional, la cual nos permite hacer una inserción y eliminación más sencilla por medio de la desconexión de punteros, un arreglo bidimensional requiere más operaciones para eliminar o insertar al inicio una fila nueva, en términos asintóticos, la lista tiene una complejidad de $O(1)$, ya que solo es generar nodo, apuntar nodo a cabeza y mover cabeza, en cambio la bidimensional requiere un arreglo auxiliar, reducir aumentar el tamaño del arreglo, y luego reinsertar cada valor 1 por 1, generando una complejidad de $O(\text{Filas} \times \text{Columnas})$, la eliminación ocurre el mismo proceso mientras que la lista tiene una complejidad de $O(n)$ ya que solo se busca la fila, se elimina, se conecta anterior con siguiente y ya queda la eliminación, el arreglo bidimensional por otro lado, sigue teniendo complejidad de $O(\text{Filas} \times \text{Columnas})$ por el mismo proceso de auxiliar, reducir, reinsertar todo.

- ¿Por qué el replay requiere una lista doblemente enlazada y no una simplemente enlazada? ¿Qué costo tiene, en su implementación, retroceder o avanzar un paso?

R/ Replay necesita ser lista doble para permitir el movimiento hacia adelante y hacia atrás de un nodo, si fuera lista simple con cada paso hacia el frente habría que moverse desde el inicio hasta $\text{actual} + 1$, o hacia atrás, moverse desde el inicio hasta $\text{actual} - 1$, en cambio con lista doble es un proceso de $O(1)$ en ambos movimientos, ya que solo es mover actual hacia su nodo anterior, o su nodo siguiente.

- ¿Cómo garantiza su implementación que la cola de eventos programados siempre mantenga al frente el evento más próximo a dispararse? ¿Qué complejidad tiene insertar un nuevo evento?

R/ El mantener al frente el evento más próximo a dispararse ahorra el tener que recorrer la cola constantemente buscando el próximo evento, básicamente hace que la ejecución sea instantánea, tomar y salir. El insertar un nuevo elemento tiene una complejidad de $O(n)$ en el peor de los casos ya que tiene que evaluar todos los nodos a ver cuando el valor representa un valor menor que algún nodo, en ese momento que encuentre algo inserta justo antes de ese nodo, generando la complejidad N .